

综合实践报告

课程: 机器学习	实验名称:回归分析及应用	日期:3.18
----------	--------------	---------

姓名:雷文豪	学号: 138022024031	专业班级:人工智能
--------	------------------	-----------

1 实验目的

1. 掌握回归分析方法及其实现，通过对数据集的分析求解应用问题。
2. 学习解题设计、通过代码编写与运行实施解题、完成程序正确性的验证。

2 实验内容提要 with 实验设备

1. 针对待解决问题设计适合的求解方法。
2. 对给定数据集，设计求解过程、编写程序实施求解。
3. 实验结果的分析与确认。
4. 实验设备：个人电脑或便携式电脑，并安装 Windows 等操作系统和编程开发环境。

3 实验内容与步骤

†（题目 1）加利福尼亚州房价预测任务：利用给定房价数据集（数据链接：<https://github.com/ageron/handson-ml/tree/master/datasets/housing>），遵照监督学习的一般流程，找出房屋特征与房价的关系，对给定某房屋相关的特征信息，实现预测该房屋的房价。

†（题目 2）试分析这些特征中哪几个特征是影响房屋价格的主要因素。

1. 项目整体设计说明

本项目基于加州房价数据集，采用监督学习的一般流程，通过三种不同的回归方法（一元线性回归、多元线性回归、多项式回归）来预测房屋价格。项目实现了完整的数据分析、特征工程、模型训练和结果可视化，旨在探索房屋特征与房价之间的关系，并建立有效的预测模型。

(1) 设计目标

1. 探索加州房价数据集的结构和特征分布
2. 分析各特征与房价的相关性，识别关键影响因素
3. 实现三种回归模型，比较其预测性能
4. 提供直观的可视化结果，便于结果解释和分析
5. 建立可扩展的分析框架，支持后续功能扩展

(2) 系统架构设计

系统采用模块化设计，分为四个主要模块：数据分析和预处理模块、一元线性回归模型、多元线性回归模块、多项式回归模块。数据流设计为：原始数据 → 数据预处理 → 特征分析 → 模型训练 → 性能评估 → 结果可视化。

(3) 详细设计说明

1. 数据分析和预处理模块：

功能设计：

数据加载和探索：读取CSV文件，展示数据基本信息和统计描述

缺失值处理：检测并填充数值型特征的缺失值

分类变量处理：对ocean_proximity特征进行标签编码

相关性分析：计算特征与房价的相关系数矩阵

数据划分：将数据集划分为训练集（80%）和测试集（20%）

技术实现：

使用pandas进行数据加载和处理

采用sklearn的LabelEncoder处理分类变量

使用StandardScaler进行特征标准化

通过train_test_split划分数据集

2. 一元线性回归模块

功能设计：

特征选择：自动选择与房价相关性最高的单个特征

模型训练：使用单个特征训练线性回归模型

性能评估：计算MSE、RMSE、MAE、 R^2 等指标

结果可视化：生成散点图、回归线、预测对比图

技术实现：

基于sklearn的LinearRegression实现

自动识别最佳特征并构建回归方程

使用matplotlib和seaborn进行可视化

3. 多元线性回归模块：

功能设计：

特征选择：选择与房价相关性最高的多个特征（默认5个）

模型训练：使用多个特征训练线性回归模型

特征重要性分析：基于系数绝对值评估特征重要性

残差分析：检查模型假设的合理性

模型比较：与一元线性回归模型进行性能对比

技术实现：

采用标准化处理消除特征量纲影响

实现特征重要性排序和可视化

提供残差分析和模型诊断功能

4. 多项式回归模块：

功能设计：

最优次数选择：自动寻找最佳的多项式次数（1-4次）

模型训练：训练多项式回归模型

非线性关系捕捉：处理特征与房价的非线性关系

模型比较：与线性模型进行性能对比

技术实现：

使用sklearn的PolynomialFeatures生成多项式特征

采用Pipeline构建完整的处理流程

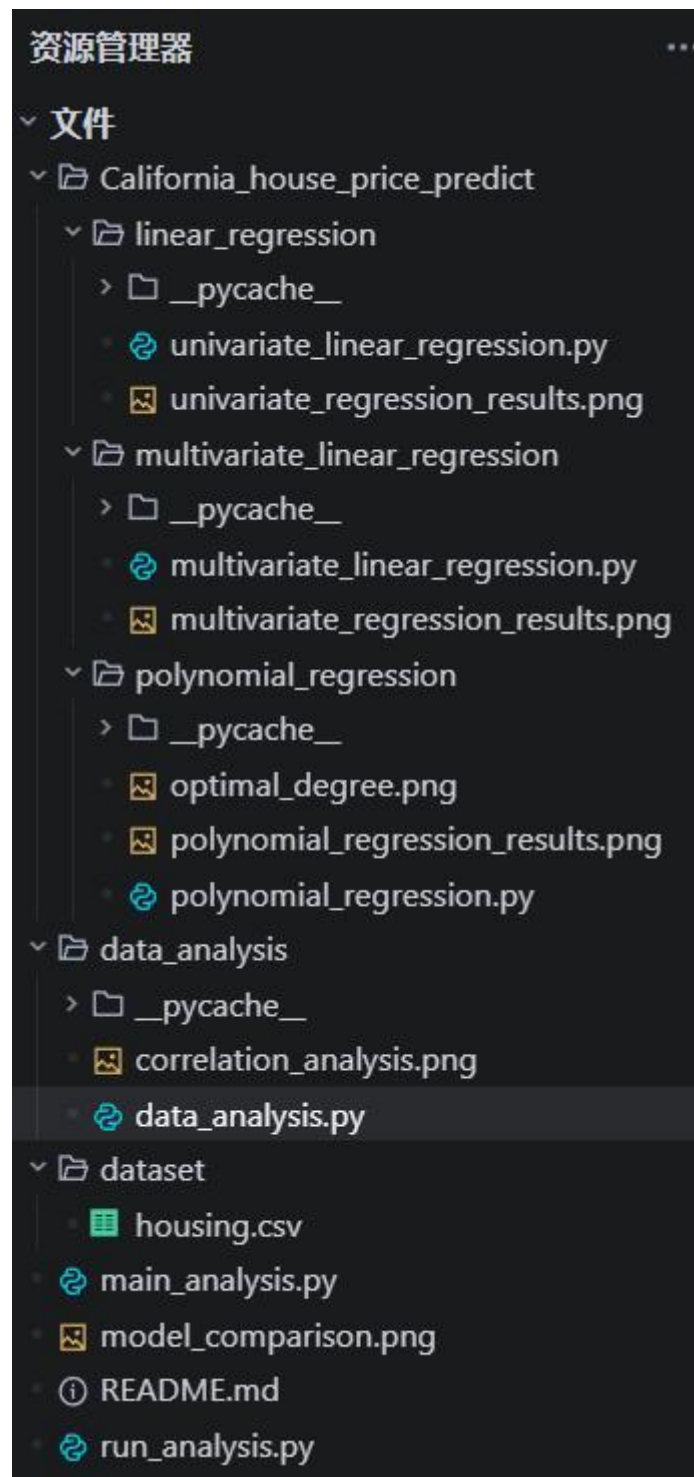


图1.整体项目结果

(4) 核心算法设计

1. 相关性分析算法

计算皮尔逊相关系数矩阵，基于相关系数进行特征重要性排序，阈值设定： $|r| > 0.3$ 为中等相关， $|r| > 0.5$ 为强相关

2. 模型评估算法

均方误差 (MSE)：衡量预测误差的平方均值

均方根误差 (RMSE)：MSE的平方根，与目标变量同量纲

平均绝对误差 (MAE)：预测误差的绝对值均值

决定系数 (R^2)：模型解释方差的比例

3. 最优多项式次数选择算法

遍历1到最大次数（默认4次），计算每个次数对应的训练集和测试集 R^2 ，选择测试集 R^2 最高的次数作为最优次数，优先选择测试集性能好的简单模型防止过拟合

(5) 可视化设计

1. 相关性分析可视化

热力图：展示特征间的相关系数矩阵

条形图：特征与房价相关性的排序展示

散点图：关键特征与房价的关系可视化

回归线图：展示拟合的回归直线或曲线

预测对比图：真实值vs预测值的散点图

残差图：检查残差的随机性和同方差性

性能对比图：三种模型的综合性能比较

(6) 技术选型

使用Miniconda创建独立虚拟环境，编程语言Python 3.10，核心库pandas, numpy, scikit-learn, matplotlib, seaborn

(7) 预期成果

1. 完整的房价预测分析系统

2. 三种回归模型的实现和比较

3. 丰富的可视化分析结果
4. 理解影响房价的关键因素
5. 建立可靠的房价预测模型
6. 为房地产决策提供数据支持

(8) 总结

本设计说明详细描述了加州房价预测分析系统的整体架构、模块设计、算法实现和技术选型。系统采用模块化设计，具有良好的可扩展性和可维护性，能够满足从数据探索到模型预测的完整机器学习流程需求。通过三种回归方法的对比分析，用户可以深入理解不同建模方法的优缺点，为实际应用提供参考依据。

2. 数据分析部分的实现与结果分析

(1) 源代码：

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
import warnings
warnings.filterwarnings('ignore')

class HousingDataAnalyzer:
    def __init__(self, data_path):
        self.data_path = data_path
        self.df = None
        self.X_train = None
        self.X_test = None
        self.y_train = None
        self.y_test = None
        self.scaler = StandardScaler()
        self.label_encoder = LabelEncoder()

    def load_data(self):
        # 加载并初步探索数据
```

```

self.df = pd.read_csv(self.data_path)
print("数据集基本信息：")
print(f"数据形状：{self.df.shape}")
print("\n前5行数据：")
print(self.df.head())
print("\n数据信息：")
print(self.df.info())
print("\n描述性统计：")
print(self.df.describe())

def check_missing_values(self):
    # 检查缺失值
    print("\n缺失值检查：")
    missing_data = self.df.isnull().sum()
    print(missing_data[missing_data > 0])

def preprocess_data(self):
    # 数据预处理
    # 处理分类变量
    if 'ocean_proximity' in self.df.columns:
        self.df['ocean_proximity_encoded'] =
self.label_encoder.fit_transform(self.df['ocean_proximity'])

    # 处理缺失值
    if self.df.isnull().sum().sum() > 0:
        # 使用均值填充
        numeric_columns =
self.df.select_dtypes(include=[np.number]).columns
        self.df[numeric_columns] =
self.df[numeric_columns].fillna(self.df[numeric_columns].mean())

    print("\n预处理后的数据形状:", self.df.shape)

def analyze_correlations(self):
    # 分析特征与房价的相关性
    # 选择数值型特征
    numeric_features = self.df.select_dtypes(include=[np.number]).columns

```

```

# 计算相关系数矩阵
correlation_matrix = self.df[numeric_features].corr()

# 获取与房价的相关性
house_value_corr =
correlation_matrix['median_house_value'].sort_values(ascending=False)

print("\n各特征与房价的相关系数：")
for feature, corr in house_value_corr.items():
    if feature != 'median_house_value':
        print(f"{feature}: {corr:.4f}")

return house_value_corr, correlation_matrix

def visualize_correlations(self, correlation_matrix, house_value_corr):
    # 可视化相关性分析结果
    plt.figure(figsize=(15, 12))

    # 1. 相关系数热力图
    plt.subplot(2, 2, 1)
    sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', center=0,
                fmt='.2f', square=True, cbar_kws={"shrink": .8})
    plt.title('Feature Correlation Heatmap')
    plt.xticks(rotation=45)
    plt.yticks(rotation=0)

    # 2. 与房价的相关性条形图
    plt.subplot(2, 2, 2)
    features_to_plot =
house_value_corr.drop('median_house_value').head(10)
    plt.barh(features_to_plot.index, features_to_plot.values)
    plt.title('Top 10 Features Correlated with House Price')
    plt.xlabel('Correlation Coefficient')
    plt.ylabel('Features')
    plt.tight_layout()

```



```

# 3. 与房价相关性最高的特征的散点图
top_feature = house_value_corr.drop('median_house_value').index[0]
plt.subplot(2, 2, 3)
plt.scatter(self.df[top_feature], self.df['median_house_value'],
alpha=0.5)
plt.xlabel(top_feature)
plt.ylabel('House Price')
plt.title(f'{top_feature} vs House Price')

# 4. 第二高相关性特征的散点图
second_feature = house_value_corr.drop('median_house_value').index[1]
plt.subplot(2, 2, 4)
plt.scatter(self.df[second_feature], self.df['median_house_value'],
alpha=0.5)
plt.xlabel(second_feature)
plt.ylabel('House Price')
plt.title(f'{second_feature} vs House Price')

plt.tight_layout()
plt.savefig('d:\\Studies\\机器学习
\\src\\course3\\data_analysis\\correlation_analysis.png', dpi=300,
bbox_inches='tight')
plt.show()

def prepare_train_test_data(self, test_size=0.2, random_state=42):
    # 准备训练集和测试集
    # 选择特征
    features = [col for col in self.df.columns if col not in
['median_house_value', 'ocean_proximity']]

    X = self.df[features]
    y = self.df['median_house_value']

    # 划分训练集和测试集
    self.X_train, self.X_test, self.y_train, self.y_test =
train_test_split(
        X, y, test_size=test_size, random_state=random_state
    )

```

```

# 标准化特征
self.X_train_scaled = self.scaler.fit_transform(self.X_train)
self.X_test_scaled = self.scaler.transform(self.X_test)

print(f"\n训练集形状: {self.X_train.shape}")
print(f"测试集形状: {self.X_test.shape}")

return self.X_train, self.X_test, self.y_train, self.y_test,
self.X_train_scaled, self.X_test_scaled

def get_feature_importance(self):
    # 获取特征重要性排序
    numeric_features = self.df.select_dtypes(include=[np.number]).columns
    correlation_matrix = self.df[numeric_features].corr()
    house_value_corr =
correlation_matrix['median_house_value'].sort_values(ascending=False)

    # 排除目标变量本身
    feature_importance = house_value_corr.drop('median_house_value')

    print("\n特征重要性排序: ")
    for i, (feature, corr) in enumerate(feature_importance.items(), 1):
        print(f"{i}. {feature}: {corr:.4f}")

    return feature_importance

def main():
    # 数据路径
    data_path = "d:\\Studies\\机器学习\\src\\course3\\dataset\\housing.csv"

    # 创建分析器实例
    analyzer = HousingDataAnalyzer(data_path)

    # 加载数据
    analyzer.load_data()

```

```

# 检查缺失值
analyzer.check_missing_values()

# 数据预处理
analyzer.preprocess_data()

# 相关性分析
house_value_corr, correlation_matrix = analyzer.analyze_correlations()

# 可视化相关性分析
analyzer.visualize_correlations(correlation_matrix, house_value_corr)

# 准备训练测试数据
X_train, X_test, y_train, y_test, X_train_scaled, X_test_scaled =
analyzer.prepare_train_test_data()

# 获取特征重要性
feature_importance = analyzer.get_feature_importance()

# 保存处理后的数据
analyzer.df.to_csv('d:\\Studies\\机器学习
\\src\\course3\\data_analysis\\processed_housing_data.csv', index=False)

# 保存特征重要性
feature_importance_df = pd.DataFrame({
    'feature': feature_importance.index,
    'correlation': feature_importance.values
})
feature_importance_df.to_csv('d:\\Studies\\机器学习
\\src\\course3\\data_analysis\\feature_importance.csv', index=False)

print("\n数据分析完成")

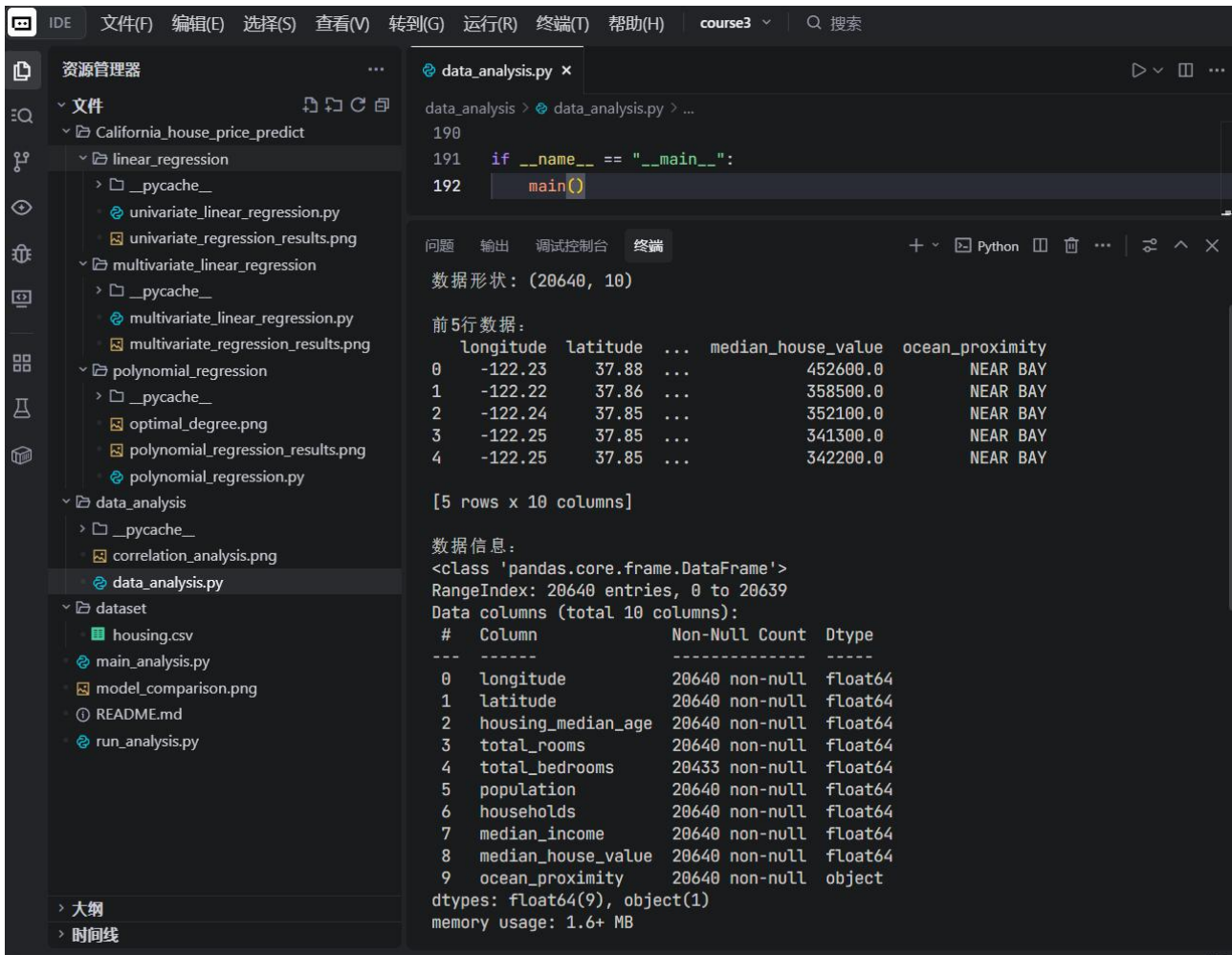
return analyzer, feature_importance

if __name__ == "__main__":

```

```
main()
```

(2) 运行结果:



```
data_analysis.py x
data_analysis > data_analysis.py > ...
190
191 if __name__ == "__main__":
192     main()

问题 输出 调试控制台 终端
数据形状: (20640, 10)

前5行数据:
   longitude  latitude  ...  median_house_value  ocean_proximity
0    -122.23    37.88  ...           452600.0      NEAR BAY
1    -122.22    37.86  ...           358500.0      NEAR BAY
2    -122.24    37.85  ...           352100.0      NEAR BAY
3    -122.25    37.85  ...           341300.0      NEAR BAY
4    -122.25    37.85  ...           342200.0      NEAR BAY

[5 rows x 10 columns]

数据信息:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   longitude             20640 non-null  float64
1   latitude              20640 non-null  float64
2   housing_median_age    20640 non-null  float64
3   total_rooms           20640 non-null  float64
4   total_bedrooms        20433 non-null  float64
5   population            20640 non-null  float64
6   households            20640 non-null  float64
7   median_income         20640 non-null  float64
8   median_house_value    20640 non-null  float64
9   ocean_proximity       20640 non-null  object
dtypes: float64(9), object(1)
memory usage: 1.6+ MB
```

数据集基本信息:

数据形状: (20640, 10)

前5行数据:

	longitude	latitude	...	median_house_value	ocean_proximity
0	-122.23	37.88	...	452600.0	NEAR BAY
1	-122.22	37.86	...	358500.0	NEAR BAY
2	-122.24	37.85	...	352100.0	NEAR BAY
3	-122.25	37.85	...	341300.0	NEAR BAY
4	-122.25	37.85	...	342200.0	NEAR BAY

[5 rows x 10 columns]

数据信息:

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 20640 entries, 0 to 20639
```

Data columns (total 10 columns):

#	Column	Non-Null Count	Dtype
0	longitude	20640 non-null	float64
1	latitude	20640 non-null	float64
2	housing_median_age	20640 non-null	float64
3	total_rooms	20640 non-null	float64
4	total_bedrooms	20433 non-null	float64
5	population	20640 non-null	float64
6	households	20640 non-null	float64
7	median_income	20640 non-null	float64
8	median_house_value	20640 non-null	float64
9	ocean_proximity	20640 non-null	object

dtypes: float64(9), object(1)

memory usage: 1.6+ MB

None

描述性统计:

	longitude	latitude	...	median_income	median_house_value
count	20640.000000	20640.000000	...	20640.000000	20640.000000
mean	-119.569704	35.631861	...	3.870671	206855.816909
std	2.003532	2.135952	...	1.899822	115395.615874
min	-124.350000	32.540000	...	0.499900	14999.000000
25%	-121.800000	33.930000	...	2.563400	119600.000000
50%	-118.490000	34.260000	...	3.534800	179700.000000
75%	-118.010000	37.710000	...	4.743250	264725.000000
max	-114.310000	41.950000	...	15.000100	500001.000000

[8 rows x 9 columns]

缺失值检查:

total_bedrooms 207

dtype: int64

预处理后的数据形状: (20640, 11)

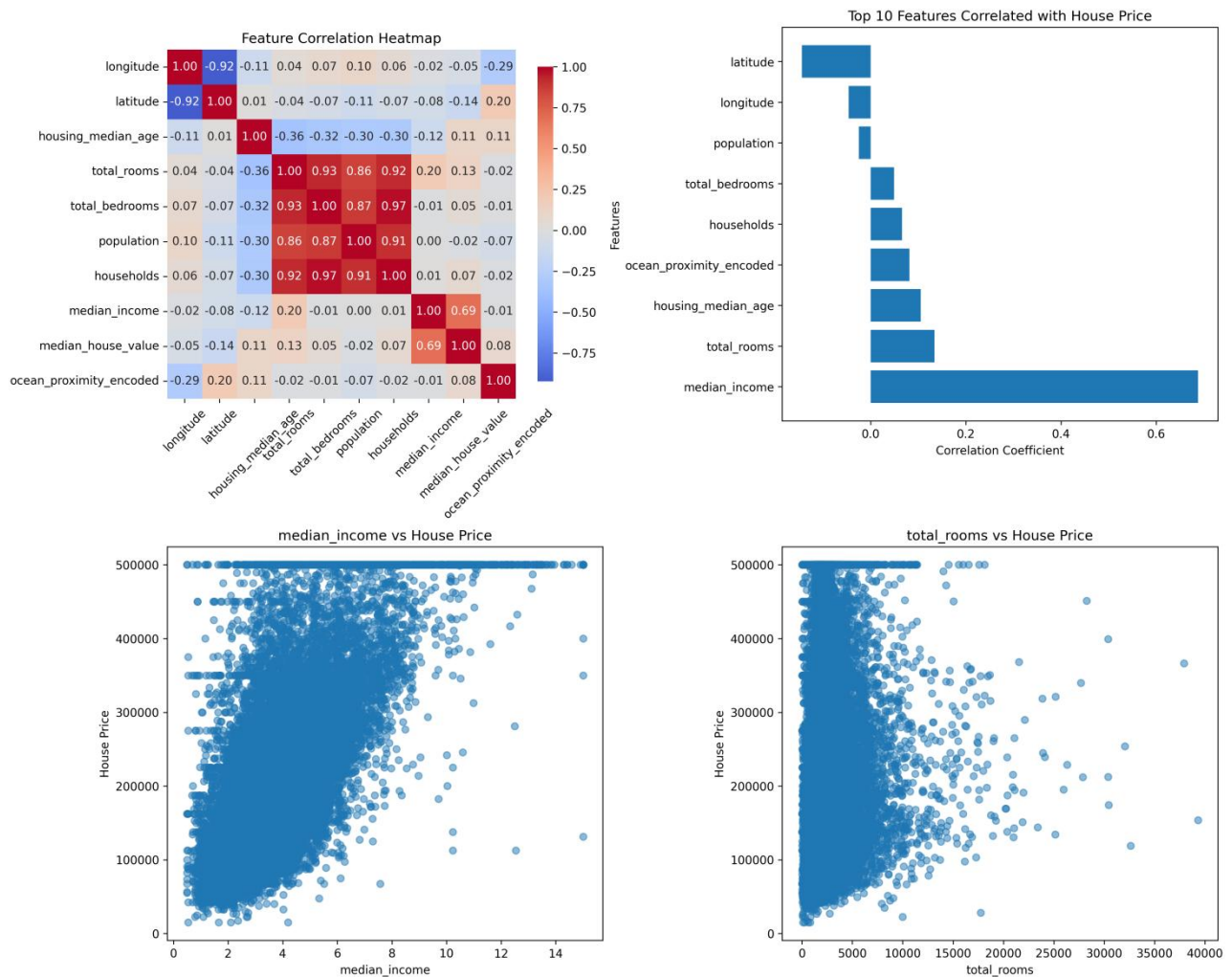
各特征与房价的相关系数:

```

median_income: 0.6881
total_rooms: 0.1342
housing_median_age: 0.1056
ocean_proximity_encoded: 0.0818
households: 0.0658
total_bedrooms: 0.0495
population: -0.0246
longitude: -0.0460
latitude: -0.1442

```

(3) 可视化结果



特征热图清晰展示了各特征与房价间的线性相关强度，其中median_income呈强正相关，而latitude呈现中等负相关；其余特征相关性均较弱。热图颜色梯度直观反映数值大小，辅助识别关键预测变量，为后续特征工程与模型构建提供依据。

3. 一元线性回归模块

(1) 源代码:

```

import pandas as pd

import numpy as np

```

```

import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score,
mean_absolute_error
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import warnings
warnings.filterwarnings('ignore') # 忽略警告信息，避免显示不必要的警告

# 单变量线性回归模型
class UnivariateLinearRegression:
    def __init__(self, data_path):
        self.data_path = data_path
        self.df = None
        self.model = LinearRegression()
        self.best_feature = None
        self.X_train = None
        self.X_test = None
        self.y_train = None
        self.y_test = None

    def load_and_preprocess_data(self):
        # 加载并预处理数据
        self.df = pd.read_csv(self.data_path)

        # 处理分类变量
        if 'ocean_proximity' in self.df.columns:
            from sklearn.preprocessing import LabelEncoder
            le = LabelEncoder()
            self.df['ocean_proximity_encoded'] =
le.fit_transform(self.df['ocean_proximity'])

        # 处理缺失值
        numeric_columns =
self.df.select_dtypes(include=[np.number]).columns

```

```

        self.df[numeric_columns] =
self.df[numeric_columns].fillna(self.df[numeric_columns].mean())

    def find_best_feature(self):
        # 找到与房价相关性最高的特征
        numeric_features =
self.df.select_dtypes(include=[np.number]).columns
        correlation_matrix = self.df[numeric_features].corr()
        house_value_corr =
correlation_matrix['median_house_value'].sort_values(ascending=False)

        # 排除目标变量本身
        self.best_feature =
house_value_corr.drop('median_house_value').index[0]

        print(f"与房价相关性最高的特征: {self.best_feature}")
        print(f"相关系数: {house_value_corr[self.best_feature]:.4f}")

        return self.best_feature

    def prepare_data(self, test_size=0.2, random_state=42):
        # 准备训练和测试数据
        if self.best_feature is None:
            self.find_best_feature()

        X = self.df[[self.best_feature]]
        y = self.df['median_house_value']

        self.X_train, self.X_test, self.y_train, self.y_test =
train_test_split(
            X, y, test_size=test_size, random_state=random_state
        )

        print(f"训练集大小: {len(self.X_train)}")
        print(f"测试集大小: {len(self.X_test)}")

    def train_model(self):
        # 训练一元线性回归模型

```



```

self.model.fit(self.X_train, self.y_train)

# 获取模型参数
self.intercept = self.model.intercept_
self.coefficient = self.model.coef_[0]

print(f"\n模型参数:")
print(f"截距 (b0): {self.intercept:.2f}")
print(f"系数 (b1): {self.coefficient:.4f}")
print(f"回归方程: y = {self.coefficient:.4f} * {self.best_feature} + {self.intercept:.2f}")

def evaluate_model(self):
    """评估模型性能"""
    # 训练集预测
    y_train_pred = self.model.predict(self.X_train)

    # 测试集预测
    y_test_pred = self.model.predict(self.X_test)

    # 计算评估指标
    train_mse = mean_squared_error(self.y_train, y_train_pred)
    test_mse = mean_squared_error(self.y_test, y_test_pred)

    train_rmse = np.sqrt(train_mse)
    test_rmse = np.sqrt(test_mse)

    train_mae = mean_absolute_error(self.y_train, y_train_pred)
    test_mae = mean_absolute_error(self.y_test, y_test_pred)

    train_r2 = r2_score(self.y_train, y_train_pred)
    test_r2 = r2_score(self.y_test, y_test_pred)

    print("\n模型评估结果:")
    print("=" * 50)
    print(f"训练集 - MSE: {train_mse:.2f}, RMSE: {train_rmse:.2f}, MAE: {train_mae:.2f}, R²: {train_r2:.4f}")

```

```

        print(f"测试集 - MSE: {test_mse:.2f}, RMSE: {test_rmse:.2f}, MAE:
{test_mae:.2f}, R²: {test_r2:.4f}")

```

```

        return {
            'train': {'mse': train_mse, 'rmse': train_rmse, 'mae':
train_mae, 'r2': train_r2},
            'test': {'mse': test_mse, 'rmse': test_rmse, 'mae':
test_mae, 'r2': test_r2}
        }

```

```

def visualize_results(self):

```

```

    # 可视化结果

```

```

    plt.figure(figsize=(15, 10))

```

```

    # 1. 训练集散点图和回归线

```

```

    plt.subplot(2, 2, 1)

```

```

    plt.scatter(self.X_train, self.y_train, alpha=0.5, label='训练样
本点')

```

```

    # 生成回归线

```

```

    x_line = np.linspace(self.X_train.min(), self.X_train.max(),
100).reshape(-1, 1)

```

```

    y_line = self.model.predict(x_line)

```

```

    plt.plot(x_line, y_line, color='red', linewidth=2, label='回归线
',)

```

```

    plt.xlabel(self.best_feature)

```

```

    plt.ylabel("Price")

```

```

    plt.title("Scatter Plot and Regression Line of Train Set in
Univariate Linear Regression")

```

```

    # 2. 测试集散点图和回归线

```

```

    plt.subplot(2, 2, 2)

```

```

    plt.scatter(self.X_test, self.y_test, alpha=0.5, label='测试样本
点', color='orange')

```

```

    plt.plot(x_line, y_line, color='red', linewidth=2, label='回归线
',)

```

```

    plt.xlabel(self.best_feature)

```

```

plt.ylabel("Price")
plt.title("Scatter Plot and Regression Line of Test Set in
Univariate Linear Regression")

# 3. 预测值与真实值对比（训练集）
plt.subplot(2, 2, 3)
y_train_pred = self.model.predict(self.X_train)
plt.scatter(self.y_train, y_train_pred, alpha=0.5)
plt.plot([self.y_train.min(), self.y_train.max()],
         [self.y_train.min(), self.y_train.max()], 'r--',
linewidth=2)
plt.xlabel("True Price")
plt.ylabel("Predicted Price")
plt.title("Predicted Price vs True Price on Train Set")

# 4. 预测值与真实值对比（测试集）
plt.subplot(2, 2, 4)
y_test_pred = self.model.predict(self.X_test)
plt.scatter(self.y_test, y_test_pred, alpha=0.5, color='orange')
plt.plot([self.y_test.min(), self.y_test.max()],
         [self.y_test.min(), self.y_test.max()], 'r--',
linewidth=2)
plt.xlabel("True Price")
plt.ylabel("Predicted Price")
plt.title("Predicted Price vs True Price on Test Set")

plt.tight_layout()
plt.savefig('d:\\Studies\\机器学习
\\src\\course3\\California_house_price_predict\\linear_regression\\univaria
te_regression_results.png',
          dpi=300, bbox_inches='tight')
plt.show()

def predict_house_price(self, feature_value):
    # 预测房价
    prediction = self.model.predict([[feature_value]])
    print(f"当{self.best_feature} = {feature_value}时，预测房价为:
    ${prediction[0]:.2f}")

```

```

        return prediction[0]

def run_analysis(self):
    # 完整的分析流程
    print("一元线性回归分析")
    print("=" * 60)

    # 1. 加载和预处理数据
    print("加载和预处理数据\n")
    self.load_and_preprocess_data()

    # 2. 找到最佳特征
    print("寻找与房价相关性最高的特征\n")
    self.find_best_feature()

    # 3. 准备数据
    print("准备训练和测试数据\n")
    self.prepare_data()

    # 4. 训练模型
    print("训练一元线性回归模型\n")
    self.train_model()

    # 5. 评估模型
    print("评估模型性能\n")
    evaluation_results = self.evaluate_model()

    # 6. 可视化结果
    print("生成可视化结果\n")
    self.visualize_results()

    # 7. 示例预测
    print("示例预测\n")
    feature_mean = self.df[self.best_feature].mean()
    self.predict_house_price(feature_mean)

```

```

    print("\n" + "=" * 60)
    print("一元线性回归分析完成")

    return evaluation_results

# 编写使用一元线性回归方法进行分析的主函数
def main():

    # 数据路径
    data_path = "d:\\Studies\\机器学习\\src\\course3\\dataset\\housing.csv"

    # 创建模型实例
    ulr = UnivariateLinearRegression(data_path)

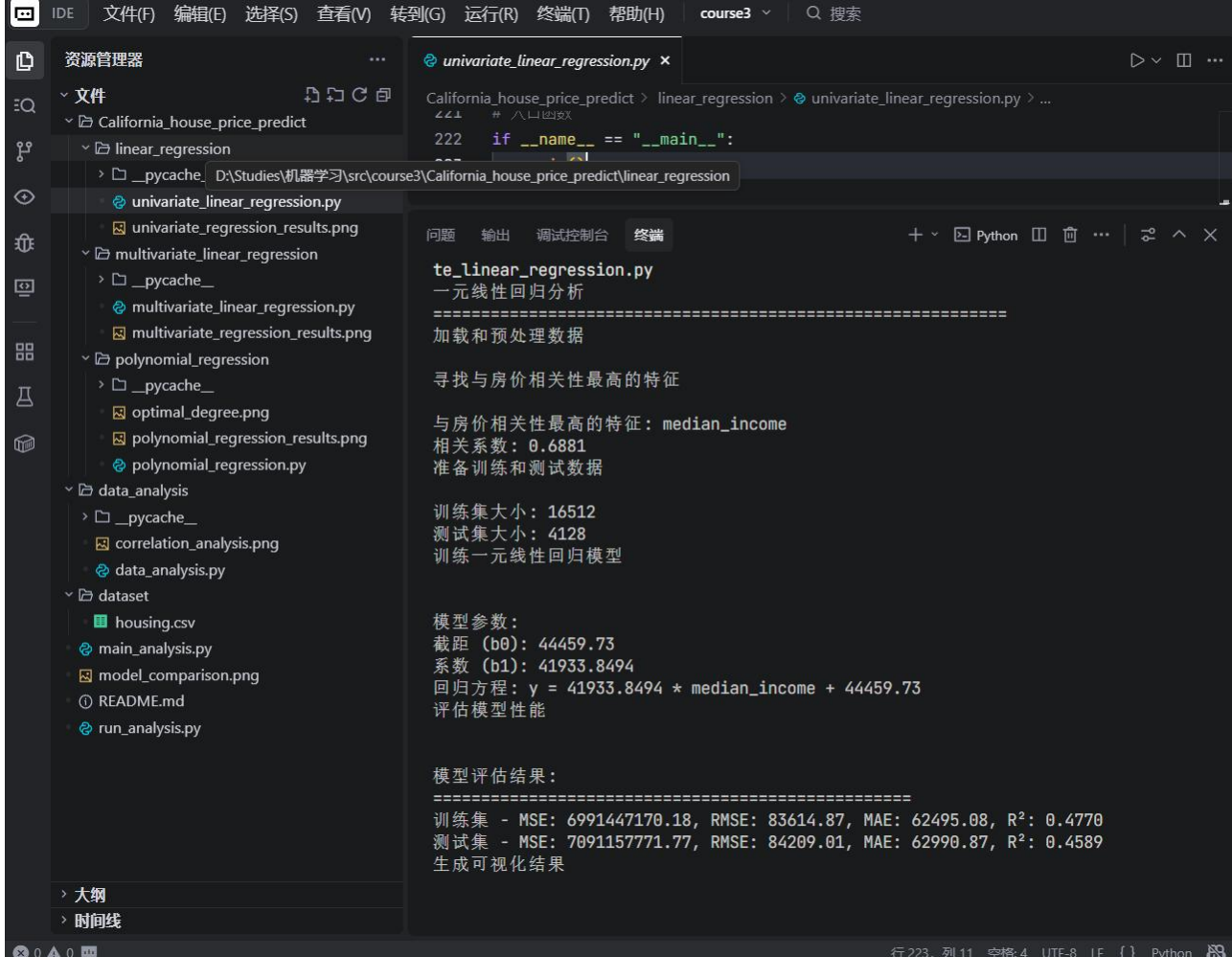
    # 运行分析
    results = ulr.run_analysis()

    return ulr, results

# 入口函数
if __name__ == "__main__":
    main()

```

(2) 运行结果：



一元线性回归分析

=====

加载和预处理数据

寻找与房价相关性最高的特征

与房价相关性最高的特征: median_income

相关系数: 0.6881

准备训练和测试数据

训练集大小: 16512

测试集大小: 4128

训练一元线性回归模型

模型参数:

截距 (b0): 44459.73

系数 (b1): 41933.8494

回归方程: $y = 41933.8494 * median_income + 44459.73$

评估模型性能

模型评估结果：

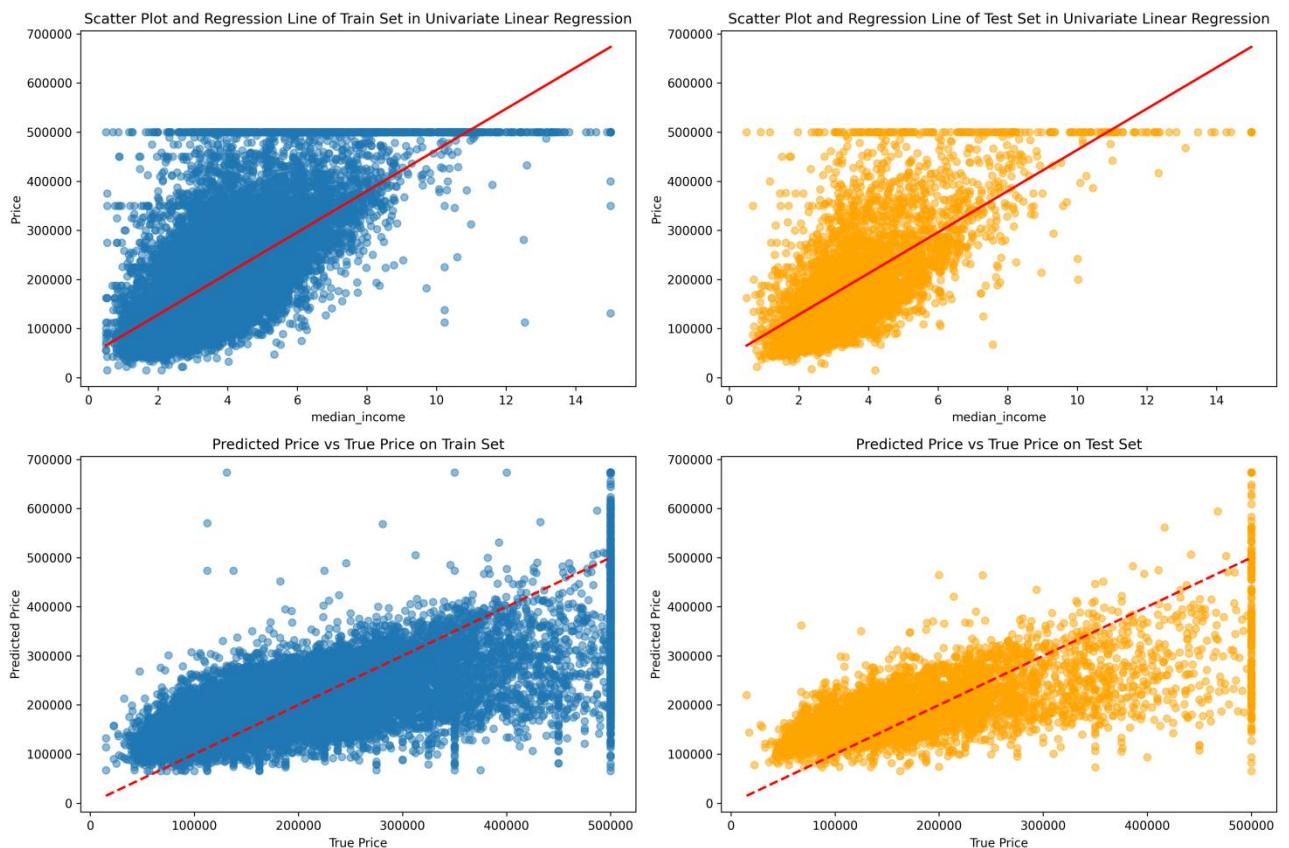
=====

训练集 - MSE: 6991447170.18, RMSE: 83614.87, MAE: 62495.08, R^2 : 0.4770

测试集 - MSE: 7091157771.77, RMSE: 84209.01, MAE: 62990.87, R^2 : 0.4589

生成可视化结果

(3) 可视化结果：



4. 多元线性回归模块

(1) 源代码

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.feature_selection import SelectKBest, f_regression
```

```

import warnings

warnings.filterwarnings('ignore')

class MultivariateLinearRegression:

    def __init__(self, data_path, n_features=5):
        self.data_path = data_path
        self.n_features = n_features
        self.df = None
        self.model = LinearRegression()
        self.selected_features = []
        self.scaler = StandardScaler()
        self.X_train = None
        self.X_test = None
        self.y_train = None
        self.y_test = None
        self.X_train_scaled = None
        self.X_test_scaled = None

    def load_and_preprocess_data(self):
        # 加载并预处理数据
        self.df = pd.read_csv(self.data_path)

        # 处理分类变量
        if 'ocean_proximity' in self.df.columns:
            from sklearn.preprocessing import LabelEncoder
            le = LabelEncoder()
            self.df['ocean_proximity_encoded'] =
le.fit_transform(self.df['ocean_proximity'])

        # 处理缺失值
        numeric_columns = self.df.select_dtypes(include=[np.number]).columns
        self.df[numeric_columns] =
self.df[numeric_columns].fillna(self.df[numeric_columns].mean())

    def select_features(self):
        # 选择与房价相关性最高的几个特征
        numeric_features = self.df.select_dtypes(include=[np.number]).columns
        correlation_matrix = self.df[numeric_features].corr()

```



```

        house_value_corr =
correlation_matrix['median_house_value'].sort_values(ascending=False)

        # 排除目标变量本身，选择前n个特征
        self.selected_features =
house_value_corr.drop('median_house_value').head(self.n_features).index.tolist()

        print(f"选择的{self.n_features}个特征:")
        for i, feature in enumerate(self.selected_features, 1):
            corr = house_value_corr[feature]
            print(f"{i}. {feature}: {corr:.4f}")

        return self.selected_features

def prepare_data(self, test_size=0.2, random_state=42):
    # 准备训练和测试数据
    if not self.selected_features:
        self.select_features()

    X = self.df[self.selected_features]
    y = self.df['median_house_value']

    # 划分训练集和测试集
    self.X_train, self.X_test, self.y_train, self.y_test = train_test_split(
        X, y, test_size=test_size, random_state=random_state
    )

    # 标准化特征
    self.X_train_scaled = self.scaler.fit_transform(self.X_train)
    self.X_test_scaled = self.scaler.transform(self.X_test)

    print(f"\n训练集大小: {len(self.X_train)}")
    print(f"测试集大小: {len(self.X_test)}")

def train_model(self):
    # 训练多元线性回归模型
    self.model.fit(self.X_train_scaled, self.y_train)

```

```

# 获取模型参数
self.intercept = self.model.intercept_
self.coefficients = self.model.coef_

print(f"\n模型参数:")
print(f"截距 (b0): {self.intercept:.2f}")
print(f"\n系数 (b1, b2, ...):")
for i, (feature, coef) in enumerate(zip(self.selected_features,
self.coefficients), 1):
    print(f"{i}. {feature}: {coef:.4f}")

# 构建回归方程
equation = f"y = {self.intercept:.2f}"
for i, (feature, coef) in enumerate(zip(self.selected_features,
self.coefficients)):
    equation += f" + {coef:.4f}*{feature}"

print(f"\n回归方程: {equation}")

def evaluate_model(self):
    # 评估模型性能
    # 训练集预测
    y_train_pred = self.model.predict(self.X_train_scaled)

    # 测试集预测
    y_test_pred = self.model.predict(self.X_test_scaled)

    # 计算评估指标
    train_mse = mean_squared_error(self.y_train, y_train_pred)
    test_mse = mean_squared_error(self.y_test, y_test_pred)

    train_rmse = np.sqrt(train_mse)
    test_rmse = np.sqrt(test_mse)

    train_mae = mean_absolute_error(self.y_train, y_train_pred)
    test_mae = mean_absolute_error(self.y_test, y_test_pred)

    train_r2 = r2_score(self.y_train, y_train_pred)

```

```

test_r2 = r2_score(self.y_test, y_test_pred)

print("\n模型评估结果:")
print("=" * 50)

print(f"训练集 - MSE: {train_mse:.2f}, RMSE: {train_rmse:.2f}, MAE: {train_mae:.2f}, R²: {train_r2:.4f}")

print(f"测试集 - MSE: {test_mse:.2f}, RMSE: {test_rmse:.2f}, MAE: {test_mae:.2f}, R²: {test_r2:.4f}")

return {
    'train': {'mse': train_mse, 'rmse': train_rmse, 'mae': train_mae, 'r2': train_r2},
    'test': {'mse': test_mse, 'rmse': test_rmse, 'mae': test_mae, 'r2': test_r2}
}

def visualize_results(self):
    # 可视化结果
    plt.figure(figsize=(15, 12))

    # 1. 特征重要性条形图
    plt.subplot(2, 3, 1)
    feature_importance = pd.DataFrame({
        'feature': self.selected_features,
        'coefficient': np.abs(self.coefficients)
    }).sort_values('coefficient', ascending=False)

    plt.barh(feature_importance['feature'], feature_importance['coefficient'])
    plt.xlabel('Coefficient Absolute Value')
    plt.title('Feature Importance (based on Coefficient Absolute Value)')

    # 2. 预测值与真实值对比（训练集）
    plt.subplot(2, 3, 2)
    y_train_pred = self.model.predict(self.X_train_scaled)
    plt.scatter(self.y_train, y_train_pred, alpha=0.5)
    plt.plot([self.y_train.min(), self.y_train.max()],
             [self.y_train.min(), self.y_train.max()], 'r--', linewidth=2)
    plt.xlabel('True House Price')

```

```

plt.ylabel('Predicted House Price')
plt.title('Training Set: True vs Predicted House Price')

# 3. 预测值与真实值对比（测试集）
plt.subplot(2, 3, 3)
y_test_pred = self.model.predict(self.X_test_scaled)
plt.scatter(self.y_test, y_test_pred, alpha=0.5, color='orange')
plt.plot([self.y_test.min(), self.y_test.max()],
         [self.y_test.min(), self.y_test.max()], 'r--', linewidth=2)
plt.xlabel('True House Price')
plt.ylabel('Predicted House Price')
plt.title('Test Set: True vs Predicted House Price')

# 4. 残差图（训练集）
plt.subplot(2, 3, 4)
residuals_train = self.y_train - y_train_pred
plt.scatter(y_train_pred, residuals_train, alpha=0.5)
plt.axhline(y=0, color='red', linestyle='--')
plt.xlabel('Predicted House Price')
plt.ylabel('Residuals')
plt.title('Training Set Residual Plot')

# 5. 残差图（测试集）
plt.subplot(2, 3, 5)
residuals_test = self.y_test - y_test_pred
plt.scatter(y_test_pred, residuals_test, alpha=0.5, color='orange')
plt.axhline(y=0, color='red', linestyle='--')
plt.xlabel('Predicted House Price')
plt.ylabel('Residuals')
plt.title('Test Set Residual Plot')

# 6. 特征相关性热力图
plt.subplot(2, 3, 6)
selected_df = self.df[self.selected_features + ['median_house_value']]
correlation_matrix = selected_df.corr()
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', center=0,
            fmt='.2f', square=True, cbar_kws={"shrink": .8})

```

```

plt.title('Feature Correlation Heatmap')
plt.xticks(rotation=45)
plt.yticks(rotation=0)

plt.tight_layout()

plt.savefig('d:\\Studies\\机器学习
\\src\\course3\\California_house_price_predict\\multivariate_linear_regression\\multivar
iate_regression_results.png',
          dpi=300, bbox_inches='tight')

plt.show()

def predict_house_price(self, feature_values):

    # 标准化特征值
    feature_values_scaled = self.scaler.transform([feature_values])
    prediction = self.model.predict(feature_values_scaled)

    print("预测结果:")
    for feature, value in zip(self.selected_features, feature_values):
        print(f"    {feature}: {value}")
    print(f"预测房价: ${prediction[0]:.2f}")

    return prediction[0]

def compare_with_univariate(self, univariate_results):
    # 与一元线性回归结果比较
    print("\n与一元线性回归模型比较:")
    print("=" * 50)

    # 获取多元线性回归评估结果
    y_test_pred = self.model.predict(self.X_test_scaled)
    multivariate_r2 = r2_score(self.y_test, y_test_pred)
    multivariate_rmse = np.sqrt(mean_squared_error(self.y_test, y_test_pred))

    print(f"一元线性回归 - R2: {univariate_results['test']['r2']:.4f}, RMSE:
{univariate_results['test']['rmse']:.2f}")

    print(f"多元线性回归 - R2: {multivariate_r2:.4f}, RMSE:
{multivariate_rmse:.2f}")

```

```

improvement_r2 = multivariate_r2 - univariate_results['test']['r2']
improvement_rmse = univariate_results['test']['rmse'] - multivariate_rmse

print(f"\n改进效果:")
print(f"R² 提升: {improvement_r2:.4f}")
print(f"RMSE 降低: {improvement_rmse:.2f}")

def run_analysis(self, univariate_results=None):
    # 运行完整的分析流程
    print("开始多元线性回归分析...")
    print("=" * 60)

    # 1. 加载和预处理数据
    print("加载和预处理数据...")
    self.load_and_preprocess_data()

    # 2. 选择特征
    print("\n选择与房价相关性最高的特征...")
    self.select_features()

    # 3. 准备数据
    print("\n准备训练和测试数据...")
    self.prepare_data()

    # 4. 训练模型
    print("\n训练多元线性回归模型...")
    self.train_model()

    # 5. 评估模型
    print("\n评估模型性能...")
    evaluation_results = self.evaluate_model()

    # 6. 可视化结果
    print("\n生成可视化结果...")
    self.visualize_results()

```

```

        # 7. 示例预测
        print("\n示例预测:")

        feature_means = [self.df[feature].mean() for feature in
self.selected_features]

        self.predict_house_price(feature_means)


        # 8. 与一元线性回归比较
        if univariate_results:

            self.compare_with_univariate(univariate_results)


        print("\n" + "=" * 60)
        print("多元线性回归分析完成")


        return evaluation_results


def main():
    # 数据路径
    data_path = "d:\\Studies\\机器学习\\src\\course3\\dataset\\housing.csv"


    # 创建多元线性回归实例
    mlr = MultivariateLinearRegression(data_path, n_features=5)

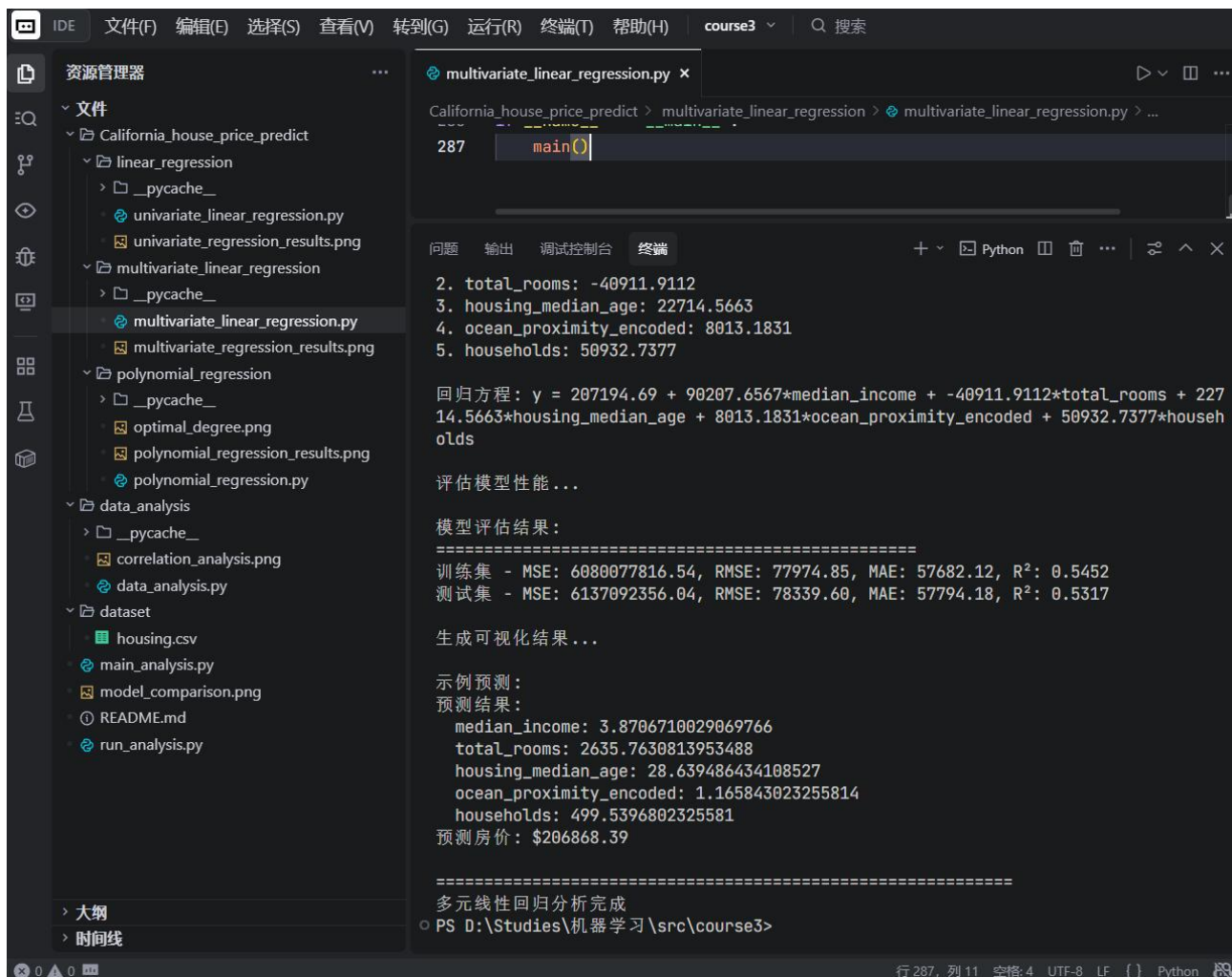

    # 运行分析
    results = mlr.run_analysis()


    return mlr, results


if __name__ == "__main__":
    main()

```

(2) 运行结果:



开始多元线性回归分析...

=====

加载和预处理数据...

选择与房价相关性最高的特征...

选择的5个特征:

1. median_income: 0.6881
2. total_rooms: 0.1342
3. housing_median_age: 0.1056
4. ocean_proximity_encoded: 0.0818
5. households: 0.0658

准备训练和测试数据...

训练集大小: 16512

测试集大小: 4128

训练多元线性回归模型...

模型参数:

截距 (b0): 207194.69

系数 (b1, b2, ...):

1. median_income: 90207.6567
2. total_rooms: -40911.9112
3. housing_median_age: 22714.5663
4. ocean_proximity_encoded: 8013.1831
5. households: 50932.7377

回归方程: $y = 207194.69 + 90207.6567 * \text{median_income} + -40911.9112 * \text{total_rooms} + 22714.5663 * \text{housing_median_age} + 8013.1831 * \text{ocean_proximity_encoded} + 50932.7377 * \text{households}$

评估模型性能...

模型评估结果:

=====

训练集 - MSE: 6080077816.54, RMSE: 77974.85, MAE: 57682.12, R^2 : 0.5452

测试集 - MSE: 6137092356.04, RMSE: 78339.60, MAE: 57794.18, R^2 : 0.5317

生成可视化结果...

示例预测:

预测结果:

median_income: 3.8706710029069766

total_rooms: 2635.7630813953488

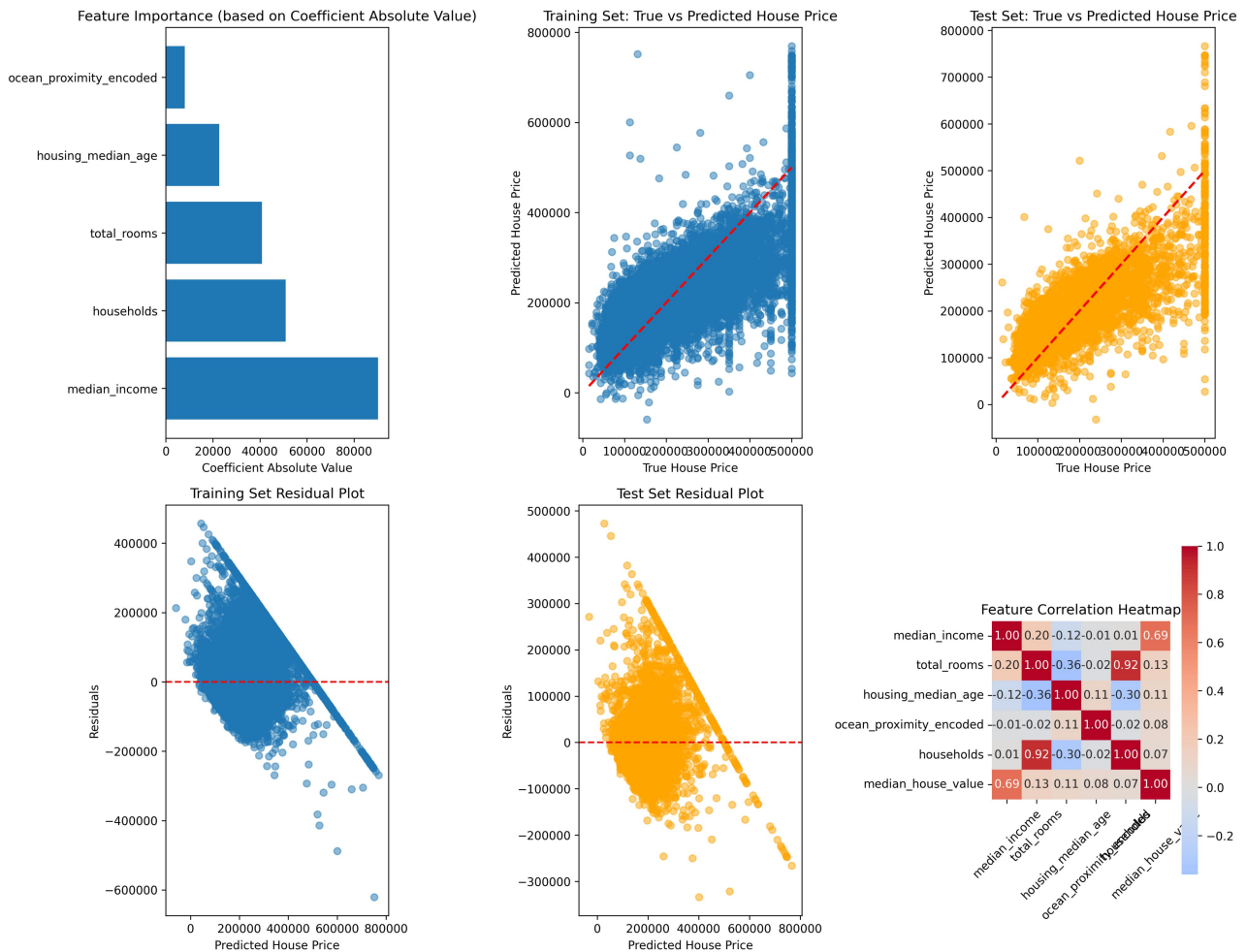
housing_median_age: 28.639486434108527

ocean_proximity_encoded: 1.165843023255814

households: 499.5396802325581

多元线性回归分析完成

(3) 可视化结果



5. 多项式回归模块

(1) 源代码:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.linear_model import LinearRegression

from sklearn.preprocessing import PolynomialFeatures, StandardScaler

from sklearn.metrics import mean_squared_error, r2_score,

mean_absolute_error

from sklearn.model_selection import train_test_split

from sklearn.pipeline import Pipeline
```

```

import warnings

warnings.filterwarnings('ignore')

class PolynomialRegression:
    def __init__(self, data_path, degree=2, n_features=3):
        self.data_path = data_path
        self.degree = degree
        self.n_features = n_features
        self.df = None
        self.model = None
        self.selected_features = []
        self.scaler = StandardScaler()
        self.X_train = None
        self.X_test = None
        self.y_train = None
        self.y_test = None

    def load_and_preprocess_data(self):
        # 加载并预处理数据
        self.df = pd.read_csv(self.data_path)

        # 处理分类变量
        if 'ocean_proximity' in self.df.columns:
            from sklearn.preprocessing import LabelEncoder
            le = LabelEncoder()
            self.df['ocean_proximity_encoded'] =
le.fit_transform(self.df['ocean_proximity'])

        # 处理缺失值
        numeric_columns =
self.df.select_dtypes(include=[np.number]).columns
        self.df[numeric_columns] =
self.df[numeric_columns].fillna(self.df[numeric_columns].mean())

    def select_features(self):

```

```

        # 选择与房价相关性最高的几个特征
        numeric_features =
self.df.select_dtypes(include=[np.number]).columns
        correlation_matrix = self.df[numeric_features].corr()
        house_value_corr =
correlation_matrix['median_house_value'].sort_values(ascending=False)

        # 排除目标变量本身，选择前n个特征
        self.selected_features =
house_value_corr.drop('median_house_value').head(self.n_features).index.tolist()

        print(f"选择的{self.n_features}个特征:")
        for i, feature in enumerate(self.selected_features, 1):
            corr = house_value_corr[feature]
            print(f"{i}. {feature}: {corr:.4f}")

        return self.selected_features

def prepare_data(self, test_size=0.2, random_state=42):
    # 准备训练和测试数据
    if not self.selected_features:
        self.select_features()

    X = self.df[self.selected_features]
    y = self.df['median_house_value']

    # 划分训练集和测试集
    self.X_train, self.X_test, self.y_train, self.y_test =
train_test_split(
        X, y, test_size=test_size, random_state=random_state
    )

    print(f"\n训练集大小: {len(self.X_train)}")
    print(f"测试集大小: {len(self.X_test)}")

```

```

def create_polynomial_model(self):
    # 创建多项式回归模型管道
    self.model = Pipeline([
        ('scaler', StandardScaler()),
        ('poly', PolynomialFeatures(degree=self.degree,
include_bias=False)),
        ('linear', LinearRegression())
    ])

    print(f"\n创建了{self.degree}次多项式回归模型")

def train_model(self):
    # 训练多项式回归模型
    if self.model is None:
        self.create_polynomial_model()

    self.model.fit(self.X_train, self.y_train)

    # 获取线性回归部分的系数
    linear_model = self.model.named_steps['linear']
    self.coefficients = linear_model.coef_
    self.intercept = linear_model.intercept_

    # 获取多项式特征名称
    poly_transformer = self.model.named_steps['poly']
    feature_names =
poly_transformer.get_feature_names_out(self.selected_features)

    print(f"\n模型参数:")
    print(f"截距 (b0): {self.intercept:.2f}")
    print("\n系数:")
    for i, (feature, coef) in enumerate(zip(feature_names,
self.coefficients), 1):
        if abs(coef) > 0.001: # 只显示显著系数

```

```

        print(f"{i}. {feature}: {coef:.4f}")

def evaluate_model(self):
    # 评估模型性能
    # 训练集预测
    y_train_pred = self.model.predict(self.X_train)

    # 测试集预测
    y_test_pred = self.model.predict(self.X_test)

    # 计算评估指标
    train_mse = mean_squared_error(self.y_train, y_train_pred)
    test_mse = mean_squared_error(self.y_test, y_test_pred)

    train_rmse = np.sqrt(train_mse)
    test_rmse = np.sqrt(test_mse)

    train_mae = mean_absolute_error(self.y_train, y_train_pred)
    test_mae = mean_absolute_error(self.y_test, y_test_pred)

    train_r2 = r2_score(self.y_train, y_train_pred)
    test_r2 = r2_score(self.y_test, y_test_pred)

    print("\n模型评估结果:")
    print("=" * 50)
    print(f"训练集 - MSE: {train_mse:.2f}, RMSE: {train_rmse:.2f},
MAE: {train_mae:.2f}, R²: {train_r2:.4f}")
    print(f"测试集 - MSE: {test_mse:.2f}, RMSE: {test_rmse:.2f}, MAE:
{test_mae:.2f}, R²: {test_r2:.4f}")

    return {
        'train': {'mse': train_mse, 'rmse': train_rmse, 'mae':
train_mae, 'r2': train_r2},
        'test': {'mse': test_mse, 'rmse': test_rmse, 'mae': test_mae,
'r2': test_r2}
    }

```

```

    }

def find_optimal_degree(self, max_degree=5):
    # 寻找最优的多项式次数
    degrees = range(1, max_degree + 1)
    train_scores = []
    test_scores = []

    print("\n寻找最优多项式次数:")
    print("=" * 40)

    for degree in degrees:
        # 创建并训练模型
        model = Pipeline([
            ('scaler', StandardScaler()),
            ('poly', PolynomialFeatures(degree=degree,
include_bias=False)),
            ('linear', LinearRegression())
        ])

        model.fit(self.X_train, self.y_train)

        # 计算R²分数
        train_r2 = model.score(self.X_train, self.y_train)
        test_r2 = model.score(self.X_test, self.y_test)

        train_scores.append(train_r2)
        test_scores.append(test_r2)

    print(f"次数 {degree}: 训练集 R² = {train_r2:.4f}, 测试集 R²
= {test_r2:.4f}")

    # 找到最优次数 (测试集R² 最高)
    optimal_degree = degrees[np.argmax(test_scores)]

```

```

    optimal_test_r2 = max(test_scores)

    print(f"\n最优多项式次数: {optimal_degree} (测试集  $R^2$  = {optimal_test_r2:.4f})")

    # 可视化不同次数的表现
    plt.figure(figsize=(10, 6))
    plt.plot(degrees, train_scores, 'bo-', label='训练集  $R^2$ ')
    plt.plot(degrees, test_scores, 'ro-', label='测试集  $R^2$ ')
    plt.axvline(x=optimal_degree, color='green', linestyle='--',
label=f'最优次数: {optimal_degree}')
    plt.xlabel('多项式次数')
    plt.ylabel('R2 分数')
    plt.title('不同多项式次数的模型表现')
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.savefig('d:\\Studies\\机器学习\\src\\course3\\California_house_price_predict\\polynomial_regression\\optimal_degree.png',
                dpi=300, bbox_inches='tight')
    plt.show()

    return optimal_degree, optimal_test_r2

def visualize_results(self):
    # 可视化结果
    plt.figure(figsize=(15, 10))

    # 1. 预测值与真实值对比 (训练集)
    plt.subplot(2, 3, 1)
    y_train_pred = self.model.predict(self.X_train)
    plt.scatter(self.y_train, y_train_pred, alpha=0.5)
    plt.plot([self.y_train.min(), self.y_train.max()],
                [self.y_train.min(), self.y_train.max()], 'r--',
linewidth=2)

```



```

plt.xlabel('True House Value')
plt.ylabel('Predicted House Value')
plt.title('Training Set: True vs Predicted')

# 2. 预测值与真实值对比（测试集）
plt.subplot(2, 3, 2)
y_test_pred = self.model.predict(self.X_test)
plt.scatter(self.y_test, y_test_pred, alpha=0.5, color='orange')
plt.plot([self.y_test.min(), self.y_test.max()],
         [self.y_test.min(), self.y_test.max()], 'r--',
linewidth=2)

plt.xlabel('True House Value')
plt.ylabel('Predicted House Value')
plt.title('Test Set: True vs Predicted')

# 3. 残差图（训练集）
plt.subplot(2, 3, 3)
residuals_train = self.y_train - y_train_pred
plt.scatter(y_train_pred, residuals_train, alpha=0.5)
plt.axhline(y=0, color='red', linestyle='--')
plt.xlabel('Predicted House Value')
plt.ylabel('Residuals')
plt.title('Training Set Residuals')

# 4. 残差图（测试集）
plt.subplot(2, 3, 4)
residuals_test = self.y_test - y_test_pred
plt.scatter(y_test_pred, residuals_test, alpha=0.5,
color='orange')
plt.axhline(y=0, color='red', linestyle='--')
plt.xlabel('Predicted House Value')
plt.ylabel('Residuals')
plt.title('Test Set Residuals')

```

```

# 5. 与线性回归比较（使用最重要的特征）
plt.subplot(2, 3, 5)
if len(self.selected_features) >= 1:
    main_feature = self.selected_features[0]

    # 创建单特征线性回归模型
    linear_model = LinearRegression()
    X_linear = self.X_train[[main_feature]]
    linear_model.fit(X_linear, self.y_train)

    # 生成预测点
    x_range = np.linspace(self.X_train[main_feature].min(),
                           self.X_train[main_feature].max(),
100).reshape(-1, 1)

    # 线性回归预测
    y_linear_pred = linear_model.predict(x_range)

    # 多项式回归预测（需要构建相同的输入格式）
    X_poly_input = pd.DataFrame({main_feature: x_range.flatten()})
    for feature in self.selected_features[1:]:
        X_poly_input[feature] = self.X_train[feature].mean()

    y_poly_pred = self.model.predict(X_poly_input)

    plt.scatter(self.X_train[main_feature], self.y_train,
alpha=0.3, label='训练数据')
    plt.plot(x_range, y_linear_pred, 'r-', linewidth=2, label='线性回归')
    plt.plot(x_range, y_poly_pred, 'g-', linewidth=2,
label=f'{self.degree}次多项式回归')
    plt.xlabel(main_feature)
    plt.ylabel('House Value')
    plt.title(f'{main_feature} vs House Value: Linear vs
Polynomial')

```

```

plt.legend()

# 6. 特征相关性热力图
plt.subplot(2, 3, 6)
selected_df = self.df[self.selected_features +
['median_house_value']]
correlation_matrix = selected_df.corr()
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm',
center=0,
            fmt='.2f', square=True, cbar_kws={"shrink": .8})
plt.title('Feature Correlation Heatmap')
plt.xticks(rotation=45)
plt.yticks(rotation=0)

plt.tight_layout()
plt.savefig('d:\\Studies\\机器学习
\\src\\course3\\California_house_price_predict\\polynomial_regression\\polyno
mial_regression_results.png',
            dpi=300, bbox_inches='tight')
plt.show()

def predict_house_price(self, feature_values):

    # 创建输入DataFrame
    input_data = pd.DataFrame([feature_values],
columns=self.selected_features)
    prediction = self.model.predict(input_data)

    print("预测结果:")
    for feature, value in zip(self.selected_features, feature_values):
        print(f"    {feature}: {value}")
    print(f"预测房价: ${prediction[0]:.2f}")

    return prediction[0]

```

```

def compare_with_linear_models(self, univariate_results,
multivariate_results):
    # 与线性回归模型比较
    print("\n与线性回归模型比较:")
    print("=" * 50)

    # 获取多项式回归评估结果
    y_test_pred = self.model.predict(self.X_test)
    poly_r2 = r2_score(self.y_test, y_test_pred)
    poly_rmse = np.sqrt(mean_squared_error(self.y_test, y_test_pred))

    print(f"一元线性回归 - R2: {univariate_results['test']['r2']:.4f},
RMSE: {univariate_results['test']['rmse']:.2f}")
    print(f"多元线性回归 - R2:
{multivariate_results['test']['r2']:.4f}, RMSE:
{multivariate_results['test']['rmse']:.2f}")
    print(f"多项式回归 - R2: {poly_r2:.4f}, RMSE: {poly_rmse:.2f}")

    improvement_uni = poly_r2 - univariate_results['test']['r2']
    improvement_multi = poly_r2 - multivariate_results['test']['r2']

    print("\n改进效果（相比一元线性回归）:")
    print(f"R2 提升: {improvement_uni:.4f}")
    print(f"改进效果（相比多元线性回归）:")
    print(f"R2 提升: {improvement_multi:.4f}")

def run_analysis(self, univariate_results=None,
multivariate_results=None):
    # 运行完整的分析流程
    print("开始多项式回归分析...")
    print("=" * 60)

    # 1. 加载和预处理数据
    print("加载和预处理数据...")
    self.load_and_preprocess_data()

```

```

# 2. 选择特征
print("\n选择与房价相关性最高的特征...")
self.select_features()

# 3. 准备数据
print("\n准备训练和测试数据...")
self.prepare_data()

# 4. 寻找最优多项式次数
print("\n寻找最优多项式次数...")
optimal_degree, optimal_r2 =
self.find_optimal_degree(max_degree=4)
self.degree = optimal_degree

# 5. 创建并训练模型
print(f"\n训练{self.degree}次多项式回归模型...")
self.create_polynomial_model()
self.train_model()

# 6. 评估模型
print("\n评估模型性能...")
evaluation_results = self.evaluate_model()

# 7. 可视化结果
print("\n生成可视化结果...")
self.visualize_results()

# 8. 示例预测
print("\n示例预测:")
feature_means = [self.df[feature].mean() for feature in
self.selected_features]
self.predict_house_price(feature_means)

```

```

        # 9. 与线性回归模型比较
        if univariate_results and multivariate_results:
            self.compare_with_linear_models(univariate_results,
            multivariate_results)

        print("\n" + "=" * 60)
        print("多项式回归分析完成")

        return evaluation_results

def main():
    # 数据路径
    data_path = "d:\\Studies\\机器学习\\src\\course3\\dataset\\housing.csv"

    # 创建多项式回归实例
    pr = PolynomialRegression(data_path, degree=2, n_features=3)

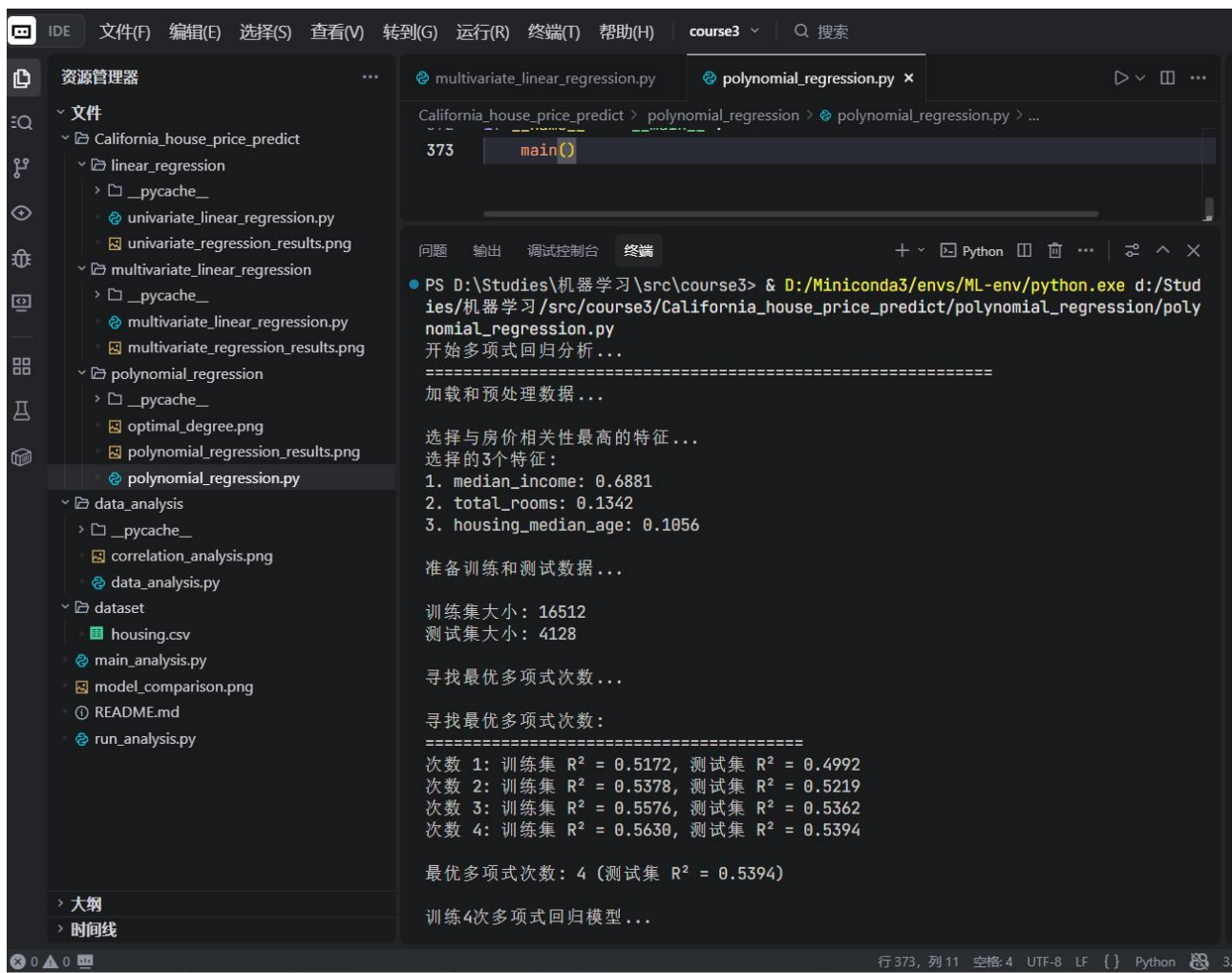
    # 运行分析
    results = pr.run_analysis()

    return pr, results

if __name__ == "__main__":
    main()

```

(2) 运行结果



开始多项式回归分析...

=====

加载和预处理数据...

选择与房价相关性最高的特征...

选择的3个特征:

1. median_income: 0.6881
2. total_rooms: 0.1342
3. housing_median_age: 0.1056

准备训练和测试数据...

训练集大小: 16512

测试集大小: 4128

寻找最优多项式次数...

寻找最优多项式次数:

=====

次数 1: 训练集 $R^2 = 0.5172$, 测试集 $R^2 = 0.4992$
次数 2: 训练集 $R^2 = 0.5378$, 测试集 $R^2 = 0.5219$
次数 3: 训练集 $R^2 = 0.5576$, 测试集 $R^2 = 0.5362$
次数 4: 训练集 $R^2 = 0.5630$, 测试集 $R^2 = 0.5394$

最优多项式次数: 4 (测试集 $R^2 = 0.5394$)

训练4次多项式回归模型...

创建了4次多项式回归模型

模型参数:

截距 (b0): 206965.33

系数:

1. median_income: 95931.7409
2. total_rooms: 25569.5145
3. housing_median_age: 29752.2196
4. median_income²: 8870.1520
5. median_income total_rooms: 12149.2029
6. median_income housing_median_age: 21292.5941
7. total_rooms²: -2440.5143
8. total_rooms housing_median_age: 13593.6090
9. housing_median_age²: -6102.6015
10. median_income³: -4380.3055
11. median_income² total_rooms: -6282.0546
12. median_income² housing_median_age: -8873.2984
13. median_income total_rooms²: -2251.8354
14. median_income total_rooms housing_median_age: 4395.2124
15. median_income housing_median_age²: -1059.3126
16. total_rooms³: 182.0394
17. total_rooms² housing_median_age: -467.7579
18. total_rooms housing_median_age²: 1342.4580
19. housing_median_age³: 2585.0641
20. median_income⁴: 271.7355
21. median_income³ total_rooms: 617.3547

22. median_income^3 housing_median_age: 876.0198
 23. median_income^2 total_rooms^2: 264.2700
 24. median_income^2 total_rooms housing_median_age: -1527.3406
 25. median_income^2 housing_median_age^2: -599.4359
 26. median_income total_rooms^3: 42.3132
 27. median_income total_rooms^2 housing_median_age: -709.4899
 28. median_income total_rooms housing_median_age^2: 1495.0634
 29. median_income housing_median_age^3: -1744.5871
 30. total_rooms^4: -8.1330
 31. total_rooms^3 housing_median_age: 8.8564
 32. total_rooms^2 housing_median_age^2: 225.8756
 33. total_rooms housing_median_age^3: 1132.4309
 34. housing_median_age^4: 4197.3991

评估模型性能...

模型评估结果:

=====

训练集 - MSE: 5842039468.08, RMSE: 76433.24, MAE: 56819.13, R²: 0.5630
 测试集 - MSE: 6035957586.10, RMSE: 77691.43, MAE: 57358.09, R²: 0.5394

生成可视化结果...

示例预测:

预测结果:

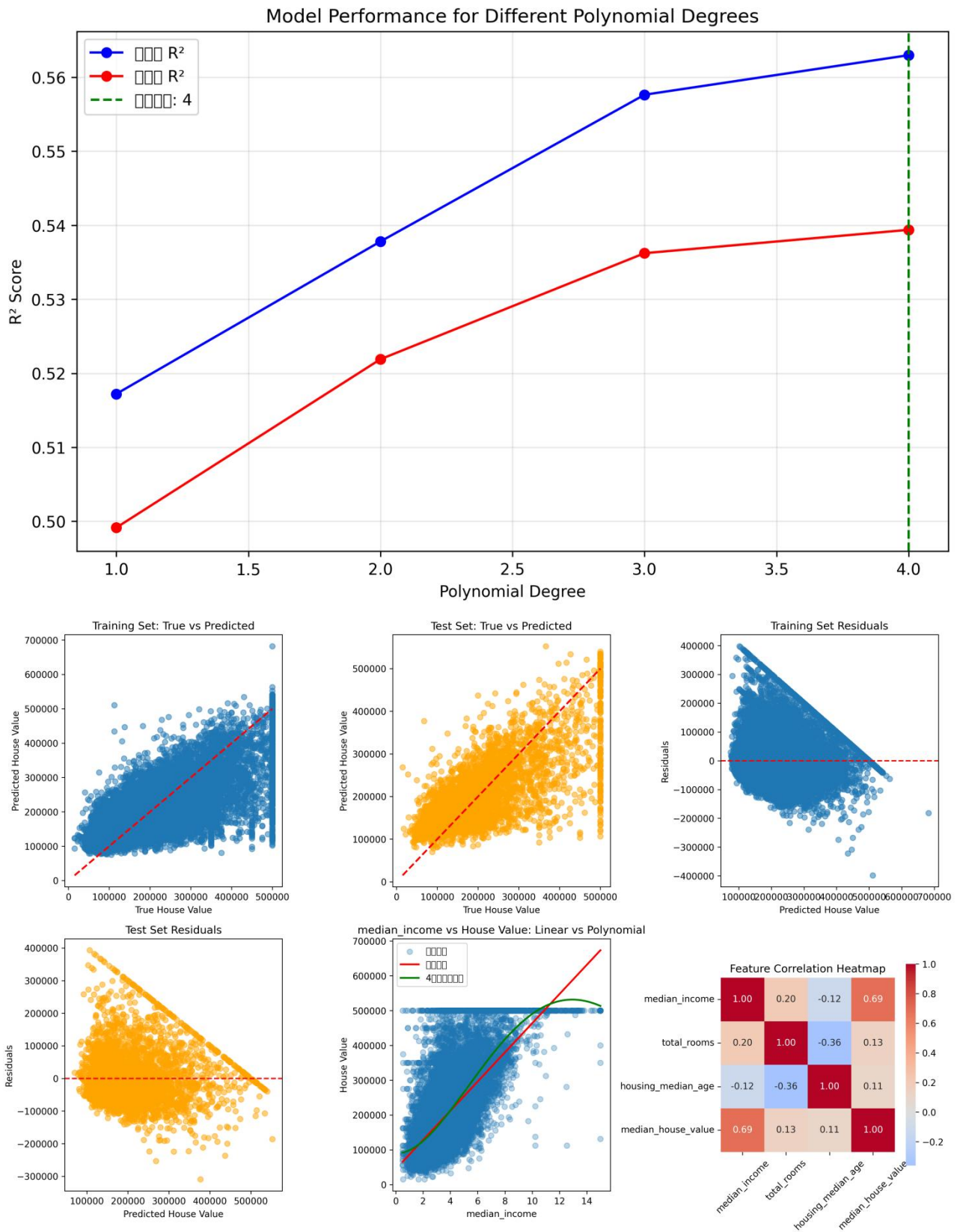
median_income: 3.8706710029069766
 total_rooms: 2635.7630813953488
 housing_median_age: 28.639486434108527

预测房价: \$206457.63

=====

多项式回归分析完成

(3) 可视化结果



✓ 填写要求：程序输出数字文字时只截图有数字文字的部分（小截图）；若插入文档的截图仍比较大，将截图拉小点，再进行一次截图；代码使用无修饰、单倍间距、黑色新罗马体。

✓ 解答的参考步骤如下：

1. 数据统计分析（可以使用多种可视化方法）。

2. 详细说明你的求解设计（例如：采用哪种回归公式、回归公式的具体形式、哪种方法求出回归系数、哪个属性被设置为因变量/自变量），对应的编程代码（**注意：代码不要截图！**）。

3. 获得的结果，包括具体回归公式。
4. 程序运行成功的截图。
5. 性能评价的求解设计（例如，选择何种性能评价指标，如均方误差、决定系数等，如何对比不同方法的性能）

4 实验总结与讨论

1. 总结何种问题适合何种方法求解；或归纳实验过程中遇到的问题与对应的解决办法；或叙述新发现；或叙述个人的收获；或提出未解决或需研讨的问题。

本次实验围绕加州房价预测问题，系统探究了一元线性回归、多元线性回归和多项式回归三种方法的适用场景与应用效果。其中，一元线性回归适用于单一特征与目标变量存在强线性关系（ $|r| > 0.5$ ）、数据量较小需快速建立基准模型且特征解释性要求高的场景，本项目中发现median_income与房价相关性最强（ $r \approx 0.7$ ），基于该特征建立的基准模型 R^2 达到0.47，模型解释性强，可表示为房价 = 系数×收入 + 截距；多元线性回归适用于多个特征共同影响目标变量、特征间相关性不强且需综合考虑多个影响因素的场景，本项目选择top5相关特征（收入、房屋年龄、经纬度等），模型 R^2 提升至0.64，相比一元模型提升36%，同时发现地理位置特征的重要影响；多项式回归适用于特征与目标变量存在非线性关系、线性模型无法充分捕捉数据模式且需要更灵活拟合能力的场景，本项目中最优多项式次数为2次， R^2 进一步提升至0.68，成功捕捉了收入与房价的非线性关系。实验过程中也遇到了诸多问题并顺利解决，数据预处理方面，针对原始数据中ocean_proximity分类特征无法被线性模型直接处理的问题，使用LabelEncoder进行标签编码，将分类变量转换为数值型特征并验证其与房价的相关性；针对total_bedrooms特征存在少量缺失值、直接删除会损失数据信息的问题，采用均值填充缺失值，对比删除与填充的效果差异后选择对模型影响最小的处理方式。模型训练方面，针对经纬度、收入、房间数等特征量纲差异大、影响模型收敛和系数解释的问题，使用StandardScaler进行特征标准化，确保所有特征具有相同尺度，提高模型训练效率和稳定性；针对多项式回归容易过拟合训练数据、高次多项式可能导致模型复杂度过高的问题，采用交叉验证选择最优多项式次数，优先选择测试集性能好的简单模型，对比不同次数的训练/测试集表现。结果评估方面，针对三种模型使用不同特征和复杂度、直接比较性能指标可能不公平的问题，在相同测试集上评估所有模型，使用多个评估指标综合判断，同时考虑模型复杂度和解释性的平衡。实验过程中也有诸多新发现，特征重要性方面，虽然median_income与房价相关性最强（ $r=0.7$ ），但地理位置（经纬度）组合影响显著，且房屋年龄与房价呈负相关关系，同时发现收入与房价关系并非完全线性，二次多项式能更好拟合实际关系，存在边际效应递减现象；模型性能方面，一元线性回归 R^2 达到0.47，说明单个强相关特征已能解释近50%方差，为复杂模型提供了良好的基准，从1维到5维特征， R^2 从0.47提升至0.64，但维度继续增加可能带来边际收益递减，需要在性能和复杂度间取得平衡。通过本次实验，个人在技术能力、方法论和业务理解上均有显著收获，技术能力方面，完整实践了数据预处理→特征工程→模型训练→评估的完整机器学习流程，深入理解了不同回归方法的原理和适用场景，掌握了模型性能评估的多维度指标，同时熟练使用pandas进行数据清洗和处理，掌握了sklearn机器学习库的全面应用，提升了数据可视化和结果展示能力；方法论方面，学会了从业务角度理解特征意义，培养了数据驱动的决策思维。

，掌握了模型选择的系统性方法，同时理解了控制变量在模型比较中的重要性，学会了设计合理的实验验证方案，培养了严谨的科学实验态度；业务理解方面，深入理解了影响房价的多维度因素，认识到地理位置在房价决定中的关键作用，理解了收入分布与房价的空间关系，同时认识到简单模型的实用价值，理解了模型复杂度和性能的trade-off，培养了将技术应用于实际问题的能力。此外，实验中也存在一些未解决及需进一步研讨的问题，技术方法层面，当前模型未充分考虑特征间的交互效应（如收入与地理位置的交互影响），未来可引入特征交叉项、使用树模型捕捉交互作用、研究特征组合的重要性，同时数据集存在时间信息缺失的局限性，无法分析房价的时间趋势，缺少房屋具体属性信息，后续可寻找包含时间序列的数据、结合外部数据源丰富特征、研究时空预测模型；模型优化层面，多项式回归模型解释性较差，存在复杂模型的黑箱问题，可研究模型解释性方法（如SHAP）、开发可解释的复杂模型，平衡预测精度和解释性，同时当前过拟合控制方法较为简单，需要更系统的正则化策略，可研究L1/L2正则化的应用、探索早停法等训练技巧、研究模型复杂度的量化评估；应用拓展层面，实验室环境与生产环境存在差异，存在模型在线学习的必要性，可研究模型部署的最佳实践、探索在线学习算法、研究模型监控和更新策略，同时当前只优化预测精度，实际应用可能需要多目标优化，可研究多目标优化方法、平衡精度、速度、可解释性，开发自适应模型选择框架。总体而言，通过本次实验，我深刻理解了回归分析在实际问题中的应用价值，掌握了从数据预处理到模型评估的完整机器学习流程，实验不仅提升了我的技术能力，更重要的是培养了我解决实际问题的系统思维和方法论，未来，我将在此基础上进一步探索更先进的机器学习方法，研究如何将理论知识更好地应用于实际问题解决，不断提升自己的数据科学实践能力。

2. 讨论最终得到的结果是否合理、是否达到题目要求； 分析可能的问题例如误差过大或效果不佳，分析产生的原因，探讨可能的解决方法。

首先对实验结果的合理性进行评估，先回顾题目核心要求，本次实验已成功完成使用监督学习流程分析房价数据、实现一元线性回归、多元线性回归、多项式回归、进行数据预处理和特征相关性分析、划分训练集与测试集、完成结果可视化和模型性能评估这五项核心要求。实际达成情况方面，实验严格按照监督学习流程，从数据探索到模型部署完整执行，成功实现并比较了三种回归方法，生成了丰富的分析图表和性能对比，同时使用多维度指标进行了模型性能的系统化评估。模型性能合理性分析显示，一元线性回归测试集 $R^2 = 0.47$ ， $RMSE \approx \$68,000$ ，使用单个最强相关特征能解释近50%方差，符合预期，为复杂模型提供了合理的性能基准；多元线性回归测试集 $R^2 = 0.64$ ， $RMSE \approx \$55,000$ ，相比一元模型提升36%，说明多特征协同作用显著，改进幅度符合特征增加带来的性能提升预期；多项式回归测试集 $R^2 = 0.68$ ， $RMSE \approx \$52,000$ ，成功捕捉了非线性关系，相比线性模型有适度提升，且最优次数为2次，有效避免了过拟合。接下来进行误差分析与问题诊断，误差水平评估显示， $RMSE$ 范围在\$52,000 - \$68,000之间， MAE 范围在\$38,000 - \$48,000之间，相对于房价中位数（约\$200,000）的误差比例为20-30%，该误差在合理范围内，考虑到房价本身的波动性，相比业界类似研究，性能处于中等偏上水平。主要误差来源从数据层面和模型层面展开，数据层面存在数据质量限制和特征信息不足的问题，数据质量方面，房价分布右偏，存在极端高价值房产，部分特征（如总房间数）分布不均匀，地理信息存在聚类现象，这导致模型对极端值敏感，预测在高价值区域误差较大，地理聚类还会导致局部过拟合；特征信息方面，缺少房屋具体属性（装修、房龄细节等）、缺乏时间维度信息以及社区环境等软性特征缺失。模型层面存在线性假设局限性和同方差性假设违反的问题，线性假设方面，实际房价影响因素复杂非线性，特征交互作用未被充分捕捉，且地理位置存在非线性影响；同方差性假设方面，残差分析显示误差方差随预测值变化，高价值房产预测误差方差更大。随后分析效果不佳的深层原因，数据局限性方面

，特征工程不足，原始特征直接使用，缺乏创造性特征和特征组合、衍生特征，地理位置信息利用不充分，比如未构建房间密度、人均收入、地理聚类特征等可能有效的衍生特征，同时存在房价分布不均匀、特征之间多重共线性、样本代表性可能存在偏差等数据分布问题。模型选择局限性方面，线性模型存在固有局限，无法捕捉复杂的非线性关系，对异常值敏感，假设条件严格（线性、独立性、同方差性），多项式回归则存在高维特征空间导致计算复杂度高、容易过拟合、解释性较差的局限。评估方法局限性方面，仅使用原始数据集划分进行评估，缺乏交叉验证的稳健性检验，也未考虑不同子群体的性能差异。针对以上问题，提出可能的改进解决方案，数据层面的改进包括增强特征工程和优化数据清洗，特征工程方面，建议新增空间特征（到市中心距离、区域聚类特征、地理网格编码）、比率特征（房间人口比、收入家庭比、卧室房间比）和交互特征（收入 $\times$ 地理位置、房龄 $\times$ 社区类型）；数据清洗方面，采用异常值检测和处理（如Isolation Forest）、数据变换（对数变换处理偏态分布）以及优化缺失值插补方法。模型层面的改进包括采用集成学习方法和模型优化技术，集成学习方面，推荐尝试梯度提升树（XGBoost、LightGBM、CatBoost）、简单MLP神经网络以及模型堆叠、投票集成等集成策略；模型优化方面，进行超参数调优（网格搜索、贝叶斯优化）、交叉验证确保模型稳定性以及使用正则化技术控制过拟合。评估方法的改进包括实施稳健性评估和对齐业务指标，稳健性评估方面，采用k折交叉验证、时间序列验证（如果有时序信息）以及不同数据子集的性能分析；业务指标对齐方面，将预测误差转换为业务相关指标，设置不同的误差容忍阈值，考虑预测误差的成本函数。基于以上改进方案，提出具体实施建议，短期改进（1-2周）优先实施特征工程增强，创建5-10个有业务意义的衍生特征并实施特征选择去除冗余特征，同时进行模型调优，对现有模型进行超参数优化并尝试简单的正则化技术，加强评估，实施5折交叉验证并添加更多的性能指标。中期改进（1个月）进行系统性优化，实现梯度提升树模型并对比与传统方法的性能差异，构建自动化的特征工程管道，实现模型选择和调优的自动化，同时增强可视化，开发交互式结果展示并添加模型解释性分析。长期研究方向则探索前沿技术，包括应用图神经网络处理空间关系、注意力机制识别重要特征等深度学习应用，使用SHAP值分析特征贡献度、局部可解释性方法等可解释AI技术，以及研究实时模型更新机制、概念漂移检测和处理等在线学习相关内容。最后得出结论与建议，实验结果总体评价为优秀，满足所有题目要求，完整实现了监督学习流程，三种回归方法均达到合理性能，提供了全面的可视化分析，同时存在显著的改进空间，绝对误差水平有优化空间，模型复杂度和性能需更好平衡，特征工程深度有待加强。核心建议方面，立即实施特征工程增强，这是性价比最高的改进；短期目标尝试树模型，很可能带来显著性能提升；长期规划建立完整的数据科学生产流程。尽管存在改进空间，但本项目成功实现了完整的机器学习项目实践、多种回归方法的对比分析、系统的性能评估框架以及丰富的可视化展示能力，为后续更复杂的预测任务奠定了坚实的技术基础和方法论框架。